Proof. Let $p^{💬}$ be a CoT Heaviside Elman RNN LM over the alphabet Σ with the regular function $\phi\colon \Delta^* \to \Sigma^*$ for an Σ-augmented alphabet Δ. Let p be the underlying RNN LM over Δ^*, that is,

$$p^{💬} = p \circ \phi^{-1} \qquad \text{and} \qquad p = p^{💬} \circ \phi$$

by Def. 3.3. We want to show the existence of a possibly non-deterministic PFSA $\mathcal{A}^{💬}$ that is weakly equivalent to $p^{💬}$. We write $p \cong p'$ to mean p is weakly equivalent to p' (cf. Def. 2.2).

By Svete and Cotterell (2023, Lem. 4.1), there exists a DPFSA $\mathcal{A} = (\Delta, Q, \delta, \lambda, \rho)$ over the augmented alphabet Δ weakly equivalent to p. This means that

$$\mathcal{A} \cong p \cong p^{💬} \circ \phi \tag{62}$$

and thus

$$p^{💬} \cong \mathcal{A} \circ \phi^{-1}. \tag{63}$$

Just like in the proof of Thm. 3.1, $\mathcal{A} \circ \phi^{-1}$ can be computed as a composition of (lifted) WFSTs that is then projected onto the input component, resulting in a PFSA weakly equivalent to $p^{💬}$. This finishes the proof. ■

Theorem 4.3. *For any regular LM, there exists a weakly equivalent CoT-augmented constant-precision transformer LM.*

Before we proceed to the full proof of Thm. 4.3, we first show the following simple but useful lemma.

Lemma F.1. *Define the following linear transformation functions:*

$$V(\mathbf{x}) \stackrel{\text{def}}{=} \mathbf{0}, \tag{64a}$$

$$O(\mathbf{x}) \stackrel{\text{def}}{=} \mathbf{0} \tag{64b}$$

for all $\mathbf{x} \in \mathbb{R}^D$. *A transformer layer* $\mathcal{L}$ *with the parameters* V *and* O *and residual connections implements the identity function irrespective of the parameters* $Q, K,$ *and* f*:*

$$\mathcal{L}(\mathbf{x}) = \mathbf{x} \tag{65}$$

for all $\mathbf{x} \in \mathbb{R}^D$.

Proof. By definition of a transformer layer (cf. Eqs. (27), (28a) and (28b)), $\mathcal{L}$ computes

$$\mathbf{a} = \texttt{Att}(\mathbf{q}, \mathbf{K}, \mathbf{V}) + \mathbf{x} = \sum_{n=1}^{N} s_n \mathbf{v}_n + \mathbf{x} = \sum_{n=1}^{N} s_n \mathbf{0} + \mathbf{x} = \mathbf{x} \tag{66a}$$

$$\mathcal{L}(\mathbf{x}) = O(\mathbf{a}) + \mathbf{a} = \mathbf{0} + \mathbf{a} = \mathbf{a} = \mathbf{x}. \tag{66b}$$

■

This allows us to show Thm. 4.3.

Proof. Let $\mathcal{A} = (\Sigma, Q, \lambda, \rho, \delta)$ be a PFSA. Like the CoT-augmented RNN in Thm. 4.2, the CoT-augmented transformer LM will work over the alphabet $\Delta \stackrel{\text{def}}{=} \underline{\Sigma}_\varepsilon \times Q$. It will start by generating a random initial state according to the initial-state distribution λ upon reading the designated padding symbol $(\textsc{bos}, q_0)$. Then, at step t of generation, it will generate the next symbol–state pair by sampling from the next-transition distribution $p\left(q_t, \underline{y}_t \mid q_{t-1}\right)$ given the state stored in the previously generated output tuple.

More formally, define $\textsc{bos}' \stackrel{\text{def}}{=} (\textsc{bos}, q_0)$ for some arbitrary but fixed $q_0 \in Q$. Further, define the static representation function

$$\mathcal{R}\left(\left(\underline{y}, q\right), t\right) \stackrel{\text{def}}{=} [\![\underline{y}, q]\!] \in \{0,1\}^{|\underline{\Sigma}_\varepsilon||Q|} \tag{67}$$

as in Eq. (57). Let $\mathcal{L}$ be the identity transformer layer from Lemma F.1 and $\mathcal{T}$ the transformer with the single layer $\mathcal{L}$. Then, by Lemma F.1,

$$\mathcal{T}(\mathcal{R})\left((\text{BOS}, q_0), (y_1, q_1), \ldots, (y_{t-1}, q_{t-1})\right) = \left([\![\text{BOS}, q_0]\!]^\top; [\![y_1, q_1]\!]^\top; \cdots; [\![y_{t-1}, q_{t-1}]\!]^\top\right) \tag{68}$$

for any t. Defining the transformation $F(\mathbf{x}) \stackrel{\text{def}}{=} \mathbf{x}$, it therefore holds that

$$\mathsf{enc}\left((\text{BOS}, q_0), (y_1, q_1), \ldots, (y_{t-1}, q_{t-1})\right) = F\left(\mathbf{x}_{t-1}^1\right) = [\![y_{t-1}, q_{t-1}]\!]. \tag{69}$$

In words, $\mathsf{enc}(\boldsymbol{y}_{<t})$ contains the current symbol–state pair of the PFSA. The CoT-augmented transformer LM can therefore sample the next symbol–state pair from the conditional distribution defined by the PFSA by setting the values of the output matrix $\mathbf{E}$ as in Thm. 4.1:

$$E_{(y',q'),(y,q)} \stackrel{\text{def}}{=} p(q', y' \mid q) \qquad q, q' \in Q, \quad y, y' \in \Sigma \tag{70a}$$

$$E_{(\varepsilon,q'),(\text{BOS},q_0)} \stackrel{\text{def}}{=} \lambda(q) \qquad q \in Q \tag{70b}$$

$$E_{(\text{EOS},q),(y,q)} \stackrel{\text{def}}{=} \rho(q) \qquad q \in Q, y \in \Sigma_\varepsilon. \tag{70c}$$

Clearly, the identical conditional probabilities defined by the CoT-augmented transformer $\mathcal{T}$ result in strings over $\Sigma_\varepsilon \times Q$ sampled with probabilities equal to the path probabilities from the PFSA, as in Thm. 4.1. Applying the transformation ϕ to the produced outputs will thus result in strings sampled from the PFSA, giving us a CoT-augmented transformer LM weakly equivalent to $\mathcal{A}$. Lastly, since all the representations in the model are position-invariant, the constructed transformer is of constant precision. ■

F.2 Probabilistic Turing Machine Language Models

RNN LMs with CoT reasoning. Next, we show that, for every PTM, its induced LM can be encoded in an unbounded precision CoT RNN LM.

Theorem 4.4. *For every LM induced by a non-deterministic probabilistic Turing machine, there exists a weakly equivalent CoT-augmented RNN LM with unbounded precision.*

Proof. We rely mainly on existing results by Nowak et al. (2023) but use the additional information afforded by the augmented output alphabet, resulting in a simpler construction and interpretation. By Nowak et al. (2023, Prop. 3.1 and Thm. 3.1), the LM families induced by PTMs and probabilistic 2PDA are weakly equivalent, so it suffices to show that a CoT RNN LM can encode any given 2PDA.

We first reiterate their main result together with its condition.

Definition F.2. *A 2PDA $\mathcal{P}$ is called **Σ-deterministic** if, for any current state q any top symbol on its first stack γ and any output symbol from $\overline{\Sigma}_\varepsilon$, there is at most one transition with non-zero weight.*

Theorem F.3. *Nowak et al. (2023, Thm 3.2) Every Σ-deterministic probabilistic 2PDA can be encoded in an RNN LM that can output empty tokens ε.*

Nowak et al. (2023, Thm. 3.2) shows that an RNN LM with output alphabet Σ_ε can simulate any probabilistic 2PDA over Σ if it is Σ-deterministic. Let $\mathcal{P}$ be an arbitrary probabilistic 2PDA, with $\mathcal{P} = (Q, \Sigma, \Gamma, \delta, q_\iota, q_\varphi)$. Now define another 2PDA $\mathcal{P} = (Q, \Delta, \Gamma, \delta', q_\iota, q_\varphi)$, which generates outputs over the extended alphabet $\Delta \stackrel{\text{def}}{=} Q \times \Gamma_\varepsilon^4 \times \Sigma_\varepsilon$. Define the transitions, add the following transitions to δ' in $\mathcal{P}'$:

$$\delta' \stackrel{\text{def}}{=} \left\{ q \xrightarrow[\gamma_2 \to \gamma_4]{\gamma, (q, \gamma_1, \gamma_2, \gamma_3, \gamma_4, y), \gamma_1 \to \gamma_3 / w} q' \mid q \xrightarrow[\gamma_2 \to \gamma_4]{\gamma, y, \gamma_1 \to \gamma_3 / w} q' \in \delta \right\} \tag{71}$$

The constructed $\mathcal{P}'$ satisfies the required Σ-determinism condition, meaning that we can construct a weakly equivalent RNN LM with ε outputs. Finally, define the following regular function $\phi\colon \Delta_\varepsilon \to \Sigma_\varepsilon$ that removes all additional information post-hoc:

$$\phi(x) \stackrel{\text{def}}{=} \begin{cases} y & \textbf{if } x = (q, \gamma_1, \gamma_2, \gamma_3, \gamma_4, y) \\ \varepsilon & \textbf{if } x = \varepsilon \end{cases} \tag{72}$$

The RNN LM therefore directly simulates runs from the 2PDA $\mathcal{P}'$. Applying ϕ to the outputs of the RNN LM results in strings in Σ^*. Since the transformation groups all execution runs that output $\boldsymbol{y} \in \Sigma^*$ and the resulting CoT-augmented LM $p^{\text{💬}}$ (cf. Def. 3.3) sums over them, we are left with a weakly equivalent LM. ■

Theorem 4.5. *For any PTM-induced LM, there exists a weakly equivalent unbounded-precision CoT-augmented Transformer LM.*

Proof. Here, we adapt the construction by Pérez et al. (2021) to the decoder-only case with prompt length (denoted by n in Pérez et al. (2021)) zero since we only care about *generating* strings from the CoT-augmented transformer LM. Moreover, rather than implementing a deterministic transition function of a Turing machine (which is simulated by a particular layer of their transformer), we implement the probabilistic transition function of a rational-valued PTM in the *sampling* step of the transformer LM. As in our previous constructions, this step endows the model with non-determinism.

At a high level, we will construct a CoT-augmented transformer LM over Σ^* that will output symbols from the augmented alphabet $\Delta \stackrel{\text{def}}{=} Q \times \Gamma \times \overline{\Sigma}_\varepsilon \times A \times A$. Here, $A \stackrel{\text{def}}{=} \{-1, 0, 1\}$, where we will for conciseness identify the action `LEFT` with -1 and the action `RIGHT` with 1. Note that we, like Pérez et al. (2021), *do not* allow the action `NOOP` in our construction, but we include the value 0 in the action set as it will be useful for the starting conditions of the constructed transformer. As explained later, such an alphabet Δ contains enough information for the transformer LM to be able to reconstruct the configuration of the PTM at every time step, and thus match its conditional probability distributions.

Notation. Let $\mathcal{M} = (Q, \Sigma, \Gamma, \delta_{\mathcal{M}}, q_\iota, q_\varphi)$ be a $\mathbb{Q}$PTM and p the LM over Σ^* it induces. In the following, we denote with $q_t \in Q$ the state of the PTM, with $v_t \in \Gamma$ the symbol written to the working tape, with a_t the action performed, with s_t the symbol read, and with y_t the symbol written to the output tape, all at time step t.

High-level idea of the construction. Before formally describing the components of the CoT-augmented transformer LM in separate lemmata, we give a high-level overview of the construction. The two-layer (single-head) transformer will, when computing $\mathsf{enc}\left(\boldsymbol{y}^{\text{💬}}_{<t}\right)$ for $\boldsymbol{y}^{\text{💬}}_{<t} \in \Delta^*$, perform the following computations, very similar to those described by Pérez et al. (2021):

1. **Input representations** ($\mathcal{R}$; Lemma F.5): The input representation function represents the input symbols $y^{\text{💬}} \in \Delta$ with a multi-hot encoding (one that contains an individual one-hot encoding of each of the components $q_t, v_{t-1}, y_{t-1}, a_{t-1}, a_{t-2}$) of the form[30,31]

$$\mathbf{r}_{\Pi}\left(y^{\text{💬}}, t\right) = \begin{pmatrix} [\![q_t]\!] \\ [\![v_{t-1}]\!] \\ [\![a_{t-1}]\!] \\ a_{t-1}, a_{t-2} \\ 0, 0 \\ 0, \mathbf{0}_{|\Gamma|} \\ 1, t+1, \frac{1}{t+1}, \frac{1}{(t+1)^2} \end{pmatrix} \tag{73}$$

   for $t \in \mathbb{N}_{\geq 0}$. The representations also include additional components (the "empty" zero values in the vector, explained below) used for processing and the positional encoding of the time step, $1, t+1, \frac{1}{t+1}, \frac{1}{(t+1)^2}$. This component is explained in more detail in Lemma F.5.

2. **Layer 1** ($\mathcal{L}_1$; Lemma F.6): The first layer uses the information about the actions performed at all previous time steps (contained in the input static representations of the CoT-augmented symbols) to

[30] In the following, green color denotes the components that are added or computed by each of the described components.

[31] Notice that the symbol $y \in \Sigma$ is not part of the internal representation, since it is not required for the simulation of the PTM. It is only used to construct the final output string stored in the output tape.